

ESCA HSE Management System

Summer Training Project — Full Scope, Technology Stack & Team Task Breakdown

Prepared for: Member 1 & Team (6 members)

Source reference: ESCA_HSE_Management_System.html (Elsewedy Cables HSE dashboard mockup, Arabic/RTL)

Nature of project: Experimental / non-production summer training project. Some enterprise-scale components from the original mockup (real CCTV computer vision, live IoT sensor networks, ERP/HRMS integrations) are scoped down and simulated with synthetic data generators rather than built against real hardware or third-party systems.

1. Project Overview

The provided HTML file is a static mockup of an enterprise Health, Safety & Environment (HSE) management platform for Elsewedy Cables. It is a starting idea, not a literal spec — it illustrates 17 functional areas (dashboard, incidents, permits, risk, training, AI/IoT, etc.) that a full production system would need. This document maps that mockup onto a realistic, buildable summer-training project for a 6-person student team: a working web frontend, a Spring Boot backend, a Flutter mobile app, and a Python AI agent with direct real-time data access that both automates admin tasks and answers admin questions in natural language.

1.1 Deliverables

- **Web dashboard** — dynamic rebuild of the mockup (React), wired to a real backend instead of hardcoded numbers.
- **Backend API** — Spring Boot services covering the core HSE modules, with authentication, roles, and an audit trail.
- **Mobile app** — Flutter app covering the highest-value on-the-go workflows (incident reporting, permit approval, inspections).
- **AI Agent** — Python service with direct read access to the database: runs scheduled automation checks and answers admin questions conversationally.

2. Team & Role Assignment (6 people)

#	Role	Person	Primary Focus
1	Backend Lead	Member 1	Core safety services: Incidents, Permits (PTW), JSA, Risk Assessment (HIRA), Auth/RBAC, Audit Trail
2	Backend / Full-stack Developer	Member 2	Operational services: Inspections, Training, HazMat, Occupational Health, Departments/Zones, Reports & Analytics

#	Role	Person	Primary Focus
3	Frontend & Mobile Developer	Member 3	React web dashboard (primary) + trimmed-scope Flutter mobile app (secondary phase)
4	AI Engineer — Automation	AI Student 1	Agent automation engine: scheduled checks, task automation, notifications
5	AI Engineer — Conversational + Simulation	AI Student 2	Agent Q&A (natural-language answers to admins) + simulated AI/IoT data feed
6	Junior Developer	Member 6	PPE & Fire Equipment Inventory module — simple CRUD, guided scope

Note on the Member 6 assignment: the PPE & Fire Equipment module is intentionally self-contained — plain CRUD (add/update/list stock items, extinguisher records, threshold flags) with no cross-module dependencies, permit logic, or security-critical rules. It's a safe, low-risk module that can be built and tested in isolation.

3. Technology Stack

Layer	Technology	Why
Web Frontend	React + Tailwind CSS + Recharts/Chart.js + Axios	Rebuilds the mockup as a dynamic SPA; charting libraries cover the KPI widgets and heatmaps
Mobile	Flutter (Dart) + Riverpod/Provider + Dio	Cross-platform, strong RTL/Arabic support, one codebase for iOS/Android
Backend Core	Java 17+, Spring Boot 3, Spring Data JPA, Spring Security (JWT), Maven	Matches the team's existing Java background; Spring Security maps directly to the RBAC/audit requirements
Database	PostgreSQL (primary) · Redis (optional caching / live counters)	Relational data fits incidents/permits/risk records; Redis optional for dashboard-speed reads
AI Agent	Python 3.11+, FastAPI, SQLAlchemy (read access), APScheduler, LLM function/tool-calling (Anthropic or OpenAI API)	FastAPI keeps the agent as its own service; tool-calling avoids letting the LLM write raw SQL
Auth	JWT issued by Spring Boot, verified by all services	Single sign-on-style trust between backend, agent, web, and mobile
Docs / Contracts	OpenAPI/Swagger (springdoc-openapi)	Auto-generated API docs so frontend/mobile/agent teams can build against a contract, not tribal knowledge
DevOps	Docker + Docker Compose, GitHub + GitHub Actions (optional CI)	Each service containerized; simple enough for a student timeline (no real Kubernetes cluster needed)

4. Full Functional Module Map

Every module from the original HTML mockup, mapped to an owner and a summer-project scope level: **Full** = real working CRUD + logic, **Simplified** = core flow only, fewer edge cases, **Simulated** = mocked/synthetic data since it needs hardware or a paid third-party system this project won't have.

Module (from mockup)	Owner	Scope	Notes
Dashboard	Member 3 (frontend)	Full	Aggregates data from all backend services
Departments & Zones	Member 2	Full	Simple reference data + linked stats
Incidents & Reports	Member 1	Full	Core module — feeds the agent's automation and Q&A
Fire Equipment	Member 6	Simplified	Register + inspection due dates, no scheduling engine
PPE Inventory	Member 6	Simplified	Stock CRUD + reorder threshold flag
Inspections & Walks	Member 2	Full	Scheduling + findings register
Risk Assessment (HIRA)	Member 1	Full	5x5 matrix logic; feeds the agent's risk alerts
Permits to Work (PTW)	Member 1	Full	Approval workflow; agent can flag/auto-expire
JSA (Job Safety Analysis)	Member 1	Simplified	Linked to permits; skip step-library tooling
HazMat & Chemicals	Member 2	Simplified	Inventory + SDS metadata, no GHS label printing
Occupational Health	Member 2	Simplified	Basic exam records; strip real medical-data compliance scope
Training & Competency	Member 2	Full	Certification expiry drives agent reminders
AI & IoT	AI Student 2	Simulated	Synthetic sensor stream (gas/noise/temp) + mocked PPE-detection events, no real cameras
Integrations (ERP/HRMS/etc.)	Member 2	Simplified	Seeded from company-provided Excel sheets instead of live ERP/HRMS/SCADA connections — see Section 6
Security & Audit	Member 1	Simplified	Spring Security roles + an append-only audit log table
System Architecture	Whole team	Simplified	Docker Compose services instead of real Kubernetes microservices
Reports & Analytics	Member 2 + Member 3	Full	Backend computes KPIs (TRIR etc.); frontend renders
Admin Automation (agent)	AI Student 1	Full	Scheduled checks: overdue permits, expiring training, open CAPAs
Admin Q&A (agent)	AI Student 2	Full	Conversational endpoint using tool-calling over live data

5. Detailed Task Breakdown Per Person

Member 1 — Backend Lead

- Design the PostgreSQL schema for Incidents, Permits, JSA, and Risk Assessment (HIRA) — the interlocking safety-critical tables.
- Build Spring Boot REST services for these four modules, following the OpenAPI contract shared with the team.
- Implement Spring Security with JWT auth and role-based access (map the mockup's 9-role RBAC matrix down to 4-5 practical roles: HSE Manager, HSE Officer, Department Manager, Worker, Auditor).
- Implement an append-only audit log (who did what, when) that every write operation feeds into.
- Expose a read-only database role/credentials for the AI agent, and define which endpoints the agent must use (vs. query directly) for write actions.
- Coordinate integration points between backend, agent, frontend, and mobile teams.

Member 2 — Backend / Full-stack Developer

- Build Spring Boot services for Inspections, Training & Competency, HazMat, Occupational Health, Departments/Zones.
- Build the Reports & Analytics endpoints (TRIR, LTIFR, severity rate — same formulas as the mockup's Reports section).
- Build simple mock endpoints simulating external integrations (e.g. a CSV-import endpoint standing in for an HR system feed).
- Support Member 3 with any backend changes needed for frontend dashboard aggregation.

Member 3 — Frontend & Mobile Developer

- Rebuild the HTML mockup as a dynamic React app: dashboard, incident register, permit board, risk matrix, training matrix — wired to live API data instead of hardcoded numbers.
- Preserve the RTL/Arabic layout from the original mockup.
- Phase 2: build a trimmed-scope Flutter mobile app covering incident reporting, permit view/approval, and inspection checklists — reusing the same backend API.
- Add a simple chat widget in the web dashboard that calls the AI agent's Q&A endpoint.

AI Student 1 — Agent: Automation Engine

- Build the Python/FastAPI agent service with a read-only SQLAlchemy connection to PostgreSQL for real-time data access.
- Implement scheduled checks (APScheduler): overdue permits, expiring training/certifications, open CAPAs past due, high-risk items unreviewed for N days.
- For any action beyond a read (e.g. auto-flagging a risk, sending a reminder), call the Spring Boot API rather than writing to the database directly — so the action is validated and captured in the audit trail.
- Design the notification mechanism (email, or an in-app notification table the frontend polls).

AI Student 2 — Agent: Conversational Q&A + AI/IoT Simulation

- Build the natural-language Q&A endpoint: define a fixed set of tool functions (e.g. `get_incidents()`, `get_permit_status()`, `get_overdue_training()`) and let the LLM pick which to call based on the admin's question, then synthesize an answer.
- Keep the agent read-only for Q&A — it answers questions, it doesn't take actions from a chat message.

- Build a synthetic data generator simulating the mockup's IoT sensors (gas, noise, temperature) and mocked computer-vision PPE-detection events, feeding believable live numbers into the dashboard without real cameras or sensors.

Member 6 — Junior Developer

- Build the PPE Inventory module: CRUD for stock items (name, code, balance, reorder threshold, consumption rate) with a simple 'below threshold' flag.
- Build the Fire Equipment Registry: CRUD for extinguishers/hydrants with status (valid / needs refill / expired) and last-inspection date.
- Coordinate with Member 1 on schema design and Spring Boot basics as needed — this module has no dependencies on other services, so it can be built and tested in isolation.

6. Data Seeding from Company-Provided Excel Sheets

Company leadership will provide Excel sheets with real reference data (likely candidates: equipment/PPE stock, fire equipment registers, employee/training records, chemical inventory, zone/department structure, and possibly historical incident logs). This upgrades the modules that were scoped as **Simulated** integration data to real seed data, and removes some guesswork from schema design — but it doesn't change the architecture, module ownership, or tech stack described above.

6.1 What this adds to the plan

- **Schema review against real data** — before finalizing table structures in Phase 1, check the actual Excel columns against the assumed schema and adjust rather than discovering mismatches later.
- **Import/ETL task** — real company spreadsheets are rarely demo-ready (inconsistent headers, merged cells, mixed units, blank rows). Someone needs to clean and load them into PostgreSQL.
- **Owner** — Member 2, who already owns the mock-integrations work, takes this over as real Excel-import work instead (Apache POI from Spring Boot, or a one-off Python + pandas/openpyxl loading script). Member 6 imports whichever sheet maps to their own PPE/Fire Equipment module — same low-risk, self-contained profile as their CRUD work.
- **Everything else is unaffected** — the AI/IoT sensor and computer-vision simulation stays synthetic (no spreadsheet replaces live sensor/camera data); modules with no matching sheet stay on their original Full/Simplified scope.

6.2 Where it fits in the timeline

Slot the import work into Phase 1 (Setup & Design), once the sheets arrive, so schema decisions are made with real data in hand rather than retrofitted later. See the updated timeline below.

7. Suggested Timeline

Phase	Weeks	Focus
1. Setup & Design	1–2	Repo setup, DB schema (reviewed against company Excel sheets), API contract (OpenAPI), auth design, dev environments (Docker Compose), import/clean the provided sheets into PostgreSQL
2. Core Build	3–6	Backend modules (Member 1 + Member 2), agent automation + Q&A skeleton, React dashboard connected to first real endpoints, Member 6 module
3. Integration	7–8	Wire frontend to all backend modules, agent reading live data end-to-end, Flutter mobile core screens
4. Simulation & Polish	9	AI/IoT synthetic data feed, remaining mocked integrations, reports/analytics, audit trail review
5. Testing & Demo Prep	10	Bug fixing, finalize demo data (real + synthetic), rehearse the walkthrough (dashboard → incident → agent flags it → admin asks the agent about it)

8. Key Design Principle Carried Through

The one rule worth keeping visible on the wall for this team: the agent **reads** data directly from the database for speed (automation checks, answering admin questions), but **writes/actions** always go through the Spring Boot API rather than the database directly. That keeps every action the agent takes — auto-flagging a risk, expiring a permit, sending a reminder — subject to the same validation and audit trail as a human action, which is exactly what the original mockup's Security & Audit section calls for.